

Exercices à faire ressortir



Ressout les exercices

Orchestrated comprehensive distributed database solutions across four exercises

Correction de la fiche de TD - Bases de Données Distribuées

Exercice 1 — Fragmentation de Compte(NoClient, Agence, TypeCompte, Somme)

Analyse des requêtes

Requête	Attributs de sélection	Attributs de projection
R1	TypeCompte = 'courant', Somme > 100 000	NoClient, Agence
R2	Agence = 'Lausanne'	(toute la ligne)
R3	Agence = 'Genève', TypeCompte = 'courant'	NoClient, Somme

Fragmentation horizontale

Les prédicats de sélection portent sur **Agence** et **TypeCompte**. On définit les prédicats simples :

- p1 : Agence = 'Lausanne'
- p2 : Agence = 'Genève'
- p3 : TypeCompte = 'courant'

D'où les fragments (minterm predicates) :

- F1 = σ [Agence='Lausanne'] Compte $\rightarrow$ couvre directement **R2**

- $F2 = \sigma[\text{Agence}='Gen\grave{e}ve' \wedge \text{TypeCompte}='courant']$ Compte $\rightarrow$ couvrir directement **R3**
- $F3 = \sigma[\text{Agence}='Gen\grave{e}ve' \wedge \text{TypeCompte} \neq 'courant']$ Compte
- $F4 = \sigma[\text{Agence} \neq 'Lausanne' \wedge \text{Agence} \neq 'Gen\grave{e}ve']$ Compte (autres agences)

R1 ($\text{TypeCompte}='courant' \wedge \text{Somme} > 100000$, toutes agences confondues) se reconstruit par **union** de tous les fragments restreints par $\text{Somme} > 100000$.

Fragmentation verticale

Matrice d'affinité des attributs avec les requêtes :

Attribut	R1	R2	R3
NoClient	proj	-	proj
Agence	proj+sel	sel	sel
TypeCompte	sel	-	sel
Somme	sel	-	proj

NoClient, Agence et TypeCompte ont une forte affinité (utilisés ensemble dans toutes les requêtes pour identifier/filtrer) ; Somme est tantôt critère, tantôt résultat. On propose :

- **V1** = {NoClient, Agence, TypeCompte}
- **V2** = {NoClient, Somme} (NoClient répété comme clé de reconstruction)

R2 (qui a besoin de la ligne complète) nécessite alors une **jointure** $V1 \bowtie V2$.

Exercice 2 — Fragments mis à jour

1) INSERT INTO compte VALUES (184177, 'Genève', 'courant', 350000)

- Horizontal : tuple avec $\text{Agence}='Gen\grave{e}ve'$ et $\text{TypeCompte}='courant'$ $\rightarrow$ va dans **F2** ($\sigma[\text{Gen\grave{e}ve} \wedge \text{courant}]$).
- Vertical : un INSERT complet doit insérer une partie dans **V1** {184177, Genève, courant} et une partie dans **V2** {184177, 350000}.

2) UPDATE Compte SET somme = somme+80000 WHERE NoClient=177498 AND TypeCompte='courant'

- Horizontal : seul le fragment contenant le compte courant du client 177498 est concerné. Or l'Agence du client n'est pas connue ici $\rightarrow$ comme la fragmentation n'est

pas basée sur NoClient, il faut interroger **tous les fragments** (ou disposer d'un annuaire/catalogue global donnant la localisation du client) pour localiser puis modifier le bon fragment.

- Vertical : seul l'attribut **Somme** est modifié → seul le fragment **V2** est mis à jour (pas besoin de toucher V1).

Exercice 3 — PRODUCTION / VENTE / CLIENT / VENDEUR

Contexte : 4 sites = Tarf (Unité Centrale), Sétif (clavier), Annaba (écran), Alger (câble). Chaque site est aussi point de vente, et chaque client s'adresse au point de vente de sa ville.

a) Fragmentation proposée

PRODUCTION → **fragmentation horizontale primaire** sur COMPOSANT (chaque site ne produit qu'un seul composant ; correspond exactement à la localité des données) :

```
F_Tarf(PRODUCTION)    =  $\sigma$ [COMPOSANT='Unité Centrale'] PRODUCTION
F_Setif(PRODUCTION)   =  $\sigma$ [COMPOSANT='clavier'] PRODUCTION
F_Annaba(PRODUCTION)  =  $\sigma$ [COMPOSANT='écran'] PRODUCTION
F_Alger(PRODUCTION)   =  $\sigma$ [COMPOSANT='câble'] PRODUCTION
```

CLIENT → **fragmentation horizontale primaire** sur VILLECLIENT (chaque client est rattaché au point de vente de sa ville) :

```
F_i(CLIENT) =  $\sigma$ [VILLECLIENT = 'ville_i'] CLIENT,  i ∈ {Tarf, Sétif, Annaba, Alger}
```

VENDEUR → **fragmentation horizontale primaire** sur VILLEVENDEUR (même logique : un vendeur travaille dans un point de vente précis) :

```
F_i(VENDEUR) =  $\sigma$ [VILLEVENDEUR = 'ville_i'] VENDEUR
```

VENTE → n'a pas d'attribut géographique propre, mais référence NOMVENDEUR et NOMCLIENT. Comme une vente est enregistrée au point de vente où se trouve le vendeur, on utilise une **fragmentation horizontale dérivée**, basée sur la fragmentation de VENDEUR (jointure sur NOMVENDEUR) :

```
F_i(VENTE) = VENTE ⋈ F_i(VENDEUR)  (semi-jointure sur NOMVENDEUR)
```

b) Schéma d'allocation

Chaque fragment est alloué au site portant son nom (allocation non redondante, basée sur la localité d'usage) :

Site	Fragments stockés
Tarf	F_Tarf(PRODUCTION) [UC], F_Tarf(CLIENT), F_Tarf(VENDEUR), F_Tarf(VENTE)
Sétif	F_Setif(PRODUCTION) [clavier], F_Setif(CLIENT), F_Setif(VENDEUR), F_Setif(VENTE)
Annaba	F_Annaba(PRODUCTION) [écran], F_Annaba(CLIENT), F_Annaba(VENDEUR), F_Annaba(VENTE)
Alger	F_Alger(PRODUCTION) [câble], F_Alger(CLIENT), F_Alger(VENDEUR), F_Alger(VENTE)

c) Requêtes émises depuis Annaba

R1 — quantité disponible du produit NUMSERIE='77y6878'

SQL transparent (vue de l'utilisateur, sur la relation globale) :

sql

```
SELECT QUANTITE
FROM PRODUCTION
WHERE NUMSERIE = '77y6878';
```

Le prédicat de la requête (NUMSERIE) **ne correspond pas** à l'attribut de fragmentation (COMPOSANT) : impossible de savoir a priori sur quel site se trouve ce numéro de série.

Le système de gestion distribué doit donc **diffuser** la requête sur les 4 fragments puis faire l'union (une seule réponse non vide est attendue) :

sql

```
SELECT QUANTITE FROM PRODUCTION_Tarf WHERE NUMSERIE='77y6878'
UNION
SELECT QUANTITE FROM PRODUCTION_Setif WHERE NUMSERIE='77y6878'
UNION
SELECT QUANTITE FROM PRODUCTION_Annaba WHERE NUMSERIE='77y6878'
UNION
SELECT QUANTITE FROM PRODUCTION_Alger WHERE NUMSERIE='77y6878';
```

R2 — somme des quantités par composant et modèle

SQL transparent :

sql

```
SELECT COMPOSANT, MODELE, SUM(QUANTITE) AS QUANTITE_TOTALE
FROM PRODUCTION
GROUP BY COMPOSANT, MODELE;
```

Ici le **GROUP BY** est **compatible** avec l'attribut de fragmentation (COMPOSANT) : chaque site ne possède qu'une seule valeur de COMPOSANT, donc les groupes ne se chevauchent jamais entre sites. La requête peut être **décomposée et exécutée localement** sur chaque site puis simplement concaténée (pas de ré-agrégation nécessaire) :

sql

```
-- sur chaque site i :
SELECT COMPOSANT, MODELE, SUM(QUANTITE) AS QUANTITE_TOTALE
FROM PRODUCTION_i
GROUP BY COMPOSANT, MODELE;

-- au site coordinateur (Annaba) :
UNION ALL des 4 résultats locaux
```

Cette requête est beaucoup plus efficace que R1 car elle exploite la fragmentation.

Exercice 4 — Schéma Employé / Projet / Affectation / Rémunération

Données (extrait classique EMP/ASG/PROJ/PAY) :

- TITLE/SAL : Elect.Eng=40000, Syst.Anal=34000, Mech.Eng=27000, Programmer=24000
- PROJ : P1(150000,Montreal), P2(135000,NewYork), P3(250000,NewYork), P4(310000,Paris)

1) Fragmentation horizontale de Rémunération

Le critère naturel (montant du salaire, pertinent pour des requêtes du type « salaires ≤ seuil ») donne :

```
PAY1 = σ[SAL ≤ 30000] Rémunération → {Mech.Eng (27000), Programmer (24000)}
PAY2 = σ[SAL > 30000] Rémunération → {Elect.Eng (40000), Syst.Anal (34000)}
```

C'est une **fragmentation horizontale primaire** (le prédicat porte directement sur un attribut de la relation).

2) Fragmentation horizontale de Projet à partir de R1 et R2

Prédicats simples extraits des requêtes :

- $p1 : VILLE = 'Montreal' / p2 : VILLE = 'New York' / p3 : VILLE = 'Paris'$ (issus de R1)
- $p4 : BUDGET < 200\,000$ (issu de R2)

En générant les **minterms** et en éliminant ceux qui sont vides au vu des données :

```
PROJ1 =  $\sigma[VILLE='Montreal' \wedge BUDGET < 200000]$  Projet → P1
PROJ2 =  $\sigma[VILLE='New York' \wedge BUDGET < 200000]$  Projet → P2
PROJ3 =  $\sigma[VILLE='New York' \wedge BUDGET \geq 200000]$  Projet → P3
PROJ4 =  $\sigma[VILLE='Paris' \wedge BUDGET \geq 200000]$  Projet → P4
```

Ce découpage est **complet et minimal** par rapport aux données existantes, et permet de répondre à R1 (par union des fragments d'une même ville) et à R2 (par union des fragments avec $BUDGET < 200000$) sans accéder à des données inutiles.

3) Fragmentation d'Employé selon les fragments de Rémunération

Employé et Rémunération sont liés par l'attribut **Titre** (relation « propriétaire » = Rémunération). On applique une **fragmentation horizontale dérivée** :

```
EMP1 = Employé  $\times$  Rémunération1 (Titre  $\in \{Mech.Eng, Programmer\}$ )
      → A.Lee, J.Miller, R.Davis (E3, E4, E7)

EMP2 = Employé  $\times$  Rémunération2 (Titre  $\in \{Elect.Eng, Syst.Anal\}$ )
      → J.Doe, M.Smith, B.Casey, L.Chu, J.Jones (E1, E2, E5, E6, E8)
```

4) Choix de fragmentation pour Affectation

Affectation est liée **à la fois** à Employé (via numE) et à Projet (via numP), qui sont *toutes les deux* maintenant fragmentées. Affectation doit donc être fragmentée par **fragmentation horizontale dérivée**, mais le choix de la relation « propriétaire » dépend du type de requêtes les plus fréquentes :

- Si les requêtes joignent surtout **Employé $\bowtie$ Affectation** (ex. « quelles sont les affectations de tel type d'employé ») → dériver depuis EMP :

$ASG1 = Affectation \times EMP1$, $ASG2 = Affectation \times EMP2$

- Si les requêtes joignent surtout **Projet** ⌘ **Affectation** (ex. « quelles affectations concernent les projets de telle ville/budget ») → dériver depuis PROJ :

$$ASGi = Affectation \times PROJi, i=1..4$$

Il n'existe pas de solution unique « correcte » : c'est un **compromis de conception** basé sur le profil de requêtes (workload) le plus fréquent de l'application — dériver depuis EMP réduit le coût des requêtes EMP-ASG, dériver depuis PROJ réduit le coût des requêtes PROJ-ASG, mais on ne peut optimiser les deux simultanément sans dupliquer l'information.

